Project Presentation

2023 IWLS Contest
Machine Learning + Logic Synthesis

team members： R12943117 隋中彧
R12921045 劉虹伶
R12943094 劉建亨

NTU GIEE
Design Verification Lab

2023.10.03

Outline

- **introduction**
- **algorithm**
  - machine learning
  - simulated annealing
  - window rewriting
- **experimental result**
- **reference**



IWLS Programming Contest 2023

- Given a Boolean function as a truth table to synthesize two circuits for each truth table:
  - (1) the traditional and-inverter-graph (AIG)
  - (2) the xor-and-inverter graph (XAIG)

- **truth table (for 5PO,6PI)**

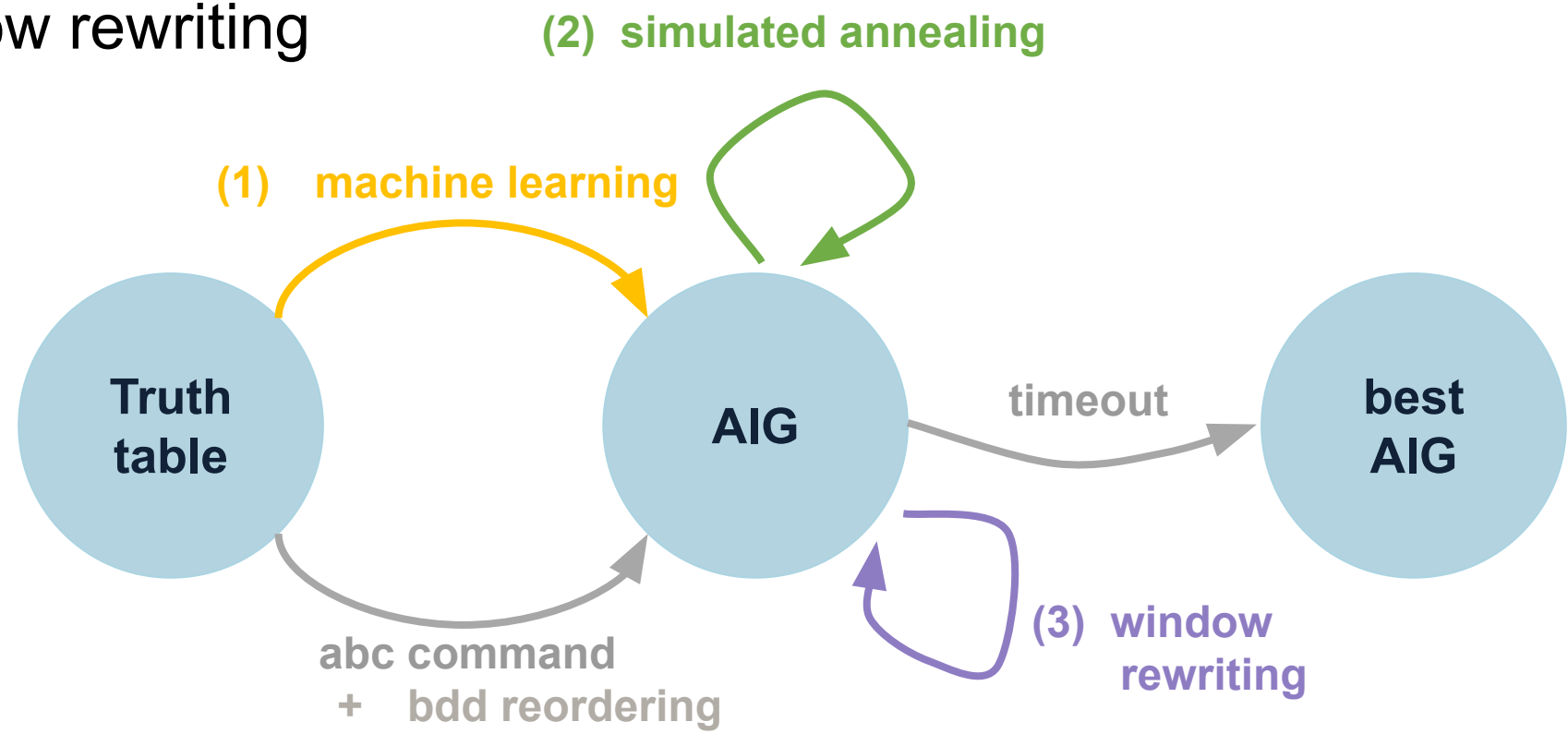


1.AIGER format file for AIG

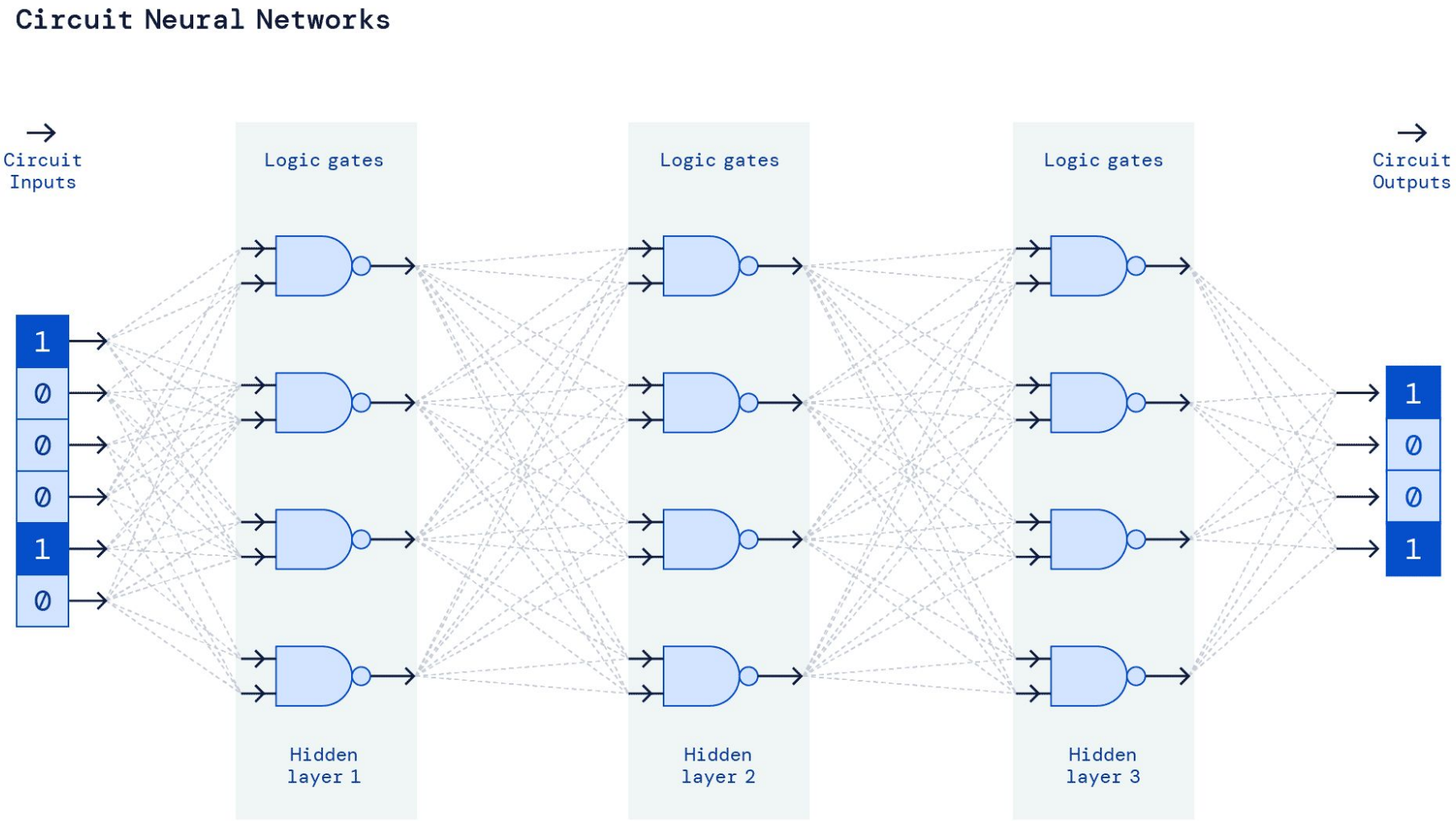
2.AIGER format file for XAIG
& use ABC with xor-gate replaced by and-gates

Algorithm

- machine learning
- simulated annealing
- window rewriting



Machine Learning - Circuit Neural Networks



Machine Learning - Circuit Neural Networks

- For the circuit:
  - we use weight matrix for each wiring between layers of gates, and use softmax to turn the matrix into probability for wirings
  - While training, we use gumbel softmax trick to pick a possible wiring according to the probability. To avoid ambiguity, we still use argmax to pick wiring while evaluation.
  - we fix the amount of AND, OR, XOR gates for each layers, only the wiring is learnable.
- we test on a truth table with 4 input with at most 4 gates is needed. We implement with 4 layers, first 2 layers with four AND, OR, XOR each, third with two and forth with one

Machine Learning - Circuit Neural Networks

- possible problems :
- There are too many possible wiring, which are all possible and suitable
  - Learning function might be plateau when far from correct circuit
  - we try : (1) initial the weight that make circuit higher probability be same as an AIG we know is correct (2) scheduled learning rate (3) fewer gates
- Boolean function might act very different with minor error
  - Learning function might be clip when near to correct circuit
  - even when we initial the weight, it might simply get to wrong place
  - we try : (1) scheduled learning rate (2) layers of weight matrix

Machine Learning - Reinforcement learning

- There are many RL model structure can be directly used in stable baseline 3. Here, I use PPO (Proximal Policy Optimization).
- We use gymnasium to implement the environment for RL
- observations
  - (1) the truth table of AIG node we remembered (here, I make it 10)
  - (2) the goal truth table
- action
  - (1) pick the AIG node we remembered as FI
  - (2) choose whether add INV on FI
  - (3) forget a node

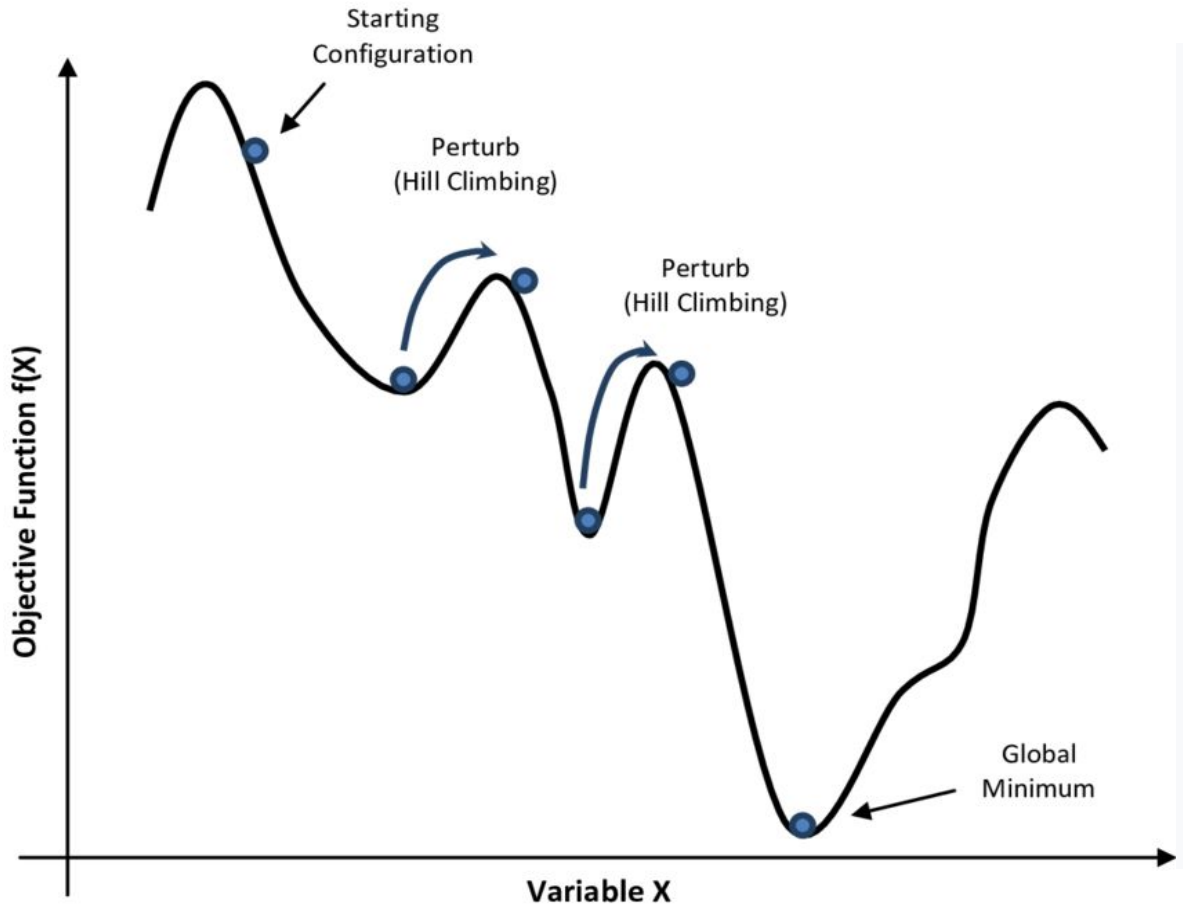
Machine Learning - Reinforcement learning

- reward
  - (1) we have a known AIG that is correct. when we compose a gate in such AIG, get 1 point
  - (2) If we compose goal, we get 2 * (limit_step - current_nAIG)
- penalty
  - (1) If there is still space to store new AIG node, don’t dump any
  - (2) don’t make 0 or 1, don’t make repeat node
- termination
  - (1) when penalty happen
  - (2) when limit_step is reached

Machine Learning - Reinforcement learning

- Result
  - We test on small circuit.
    - case 1 : #PI = 3, #PO = 1, #node <= 5
    - case 2 : #PI = 4, #PO = 1, #node <= 7
    - case 3 : #PI = 4, #PO = 2, #node <= 9 (with given AIG is not simplest)
  - Can find case 1 and case 2
    - however, can’t get better than the given AIG
  - Can only find one PO for case 3
    - Also, can’t do better than the given AIG on that PO.
    - we also try to use pre-trained (with given AIG) and train on the environment without any AIG given. It can’t do better, either.

Simulated Annealing




```
s := s0; e := E(s)
k := 0
while k < kmax and e > emin
 :
    sn := neighbour(s)
    en := E(sn)
    if random() < P(e, en, temp(k/kmax))
        s := sn; e := en
    k := k + 1
return s
```

Simulated Annealing

- resyn
  - rewrite, refactor, resubstitution
- deepsyn
- transduction, dc2
- orchestrate

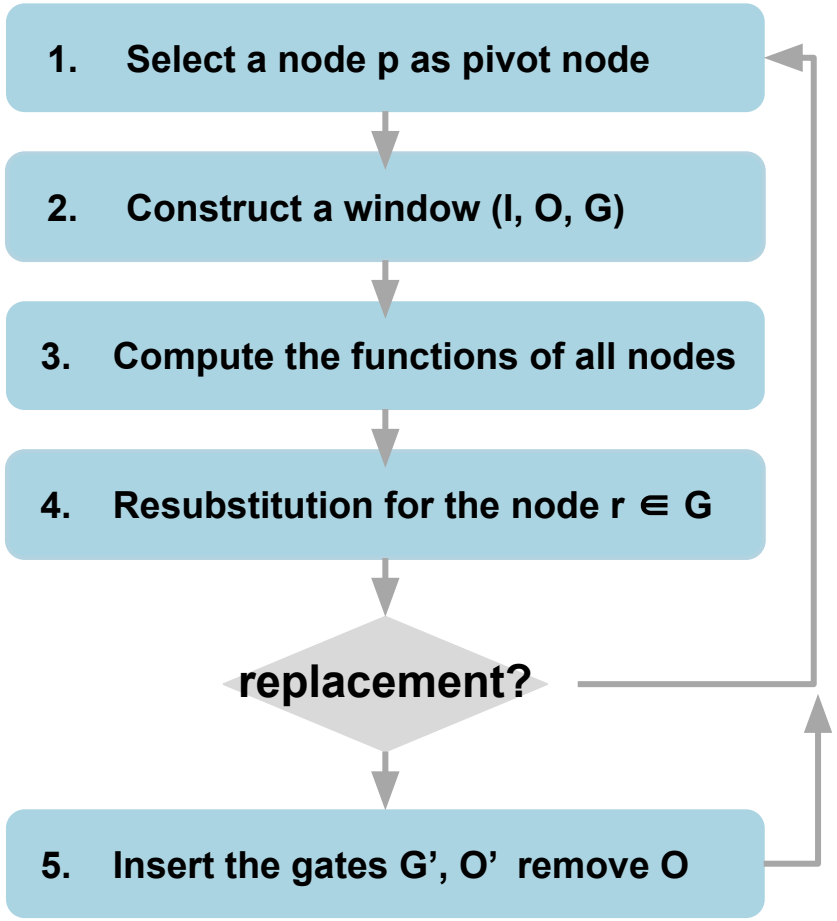
Window Rewriting

Resubstitution with Don't Cares

- Consider all or some nodes in Boolean network
  - Create window
  - Select possible fanin nodes (divisors)
  - For each candidate *subset* of divisors
    - Rule out some subsets using simulation
    - Check resubstitution feasibility using SAT
    - Compute resubstitution function using interpolation
      - A low-cost by-product of completed SAT proofs
  - Update the network if there is an improvement

divisor of r = G + I - TFO(r) - MFFC(r)

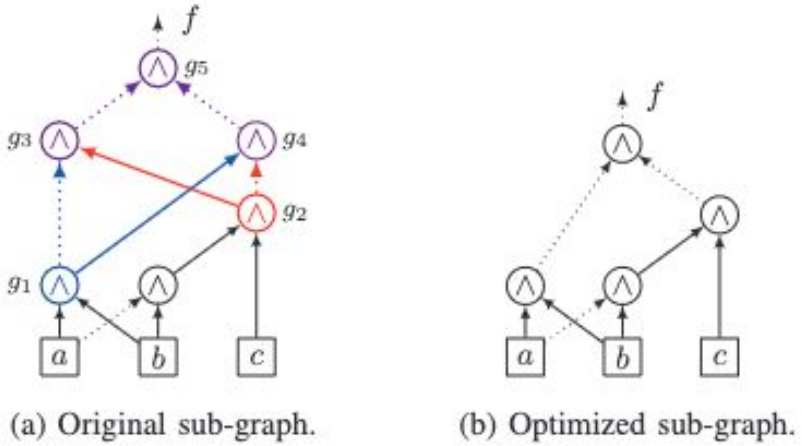
ref: lecture07



1. Reconvergence-driven windowing

Essential for don’t-care-based optimization

- **CDC-based optimization**
  The assignment fg1 = 1 and fg2 = 1 is a CDC for XOR.
  The three AND gates can be
  replaced by a single AND gate



(a) Original sub-graph. (b) Optimized sub-graph.

- **ODC-based optimization**
  v transforms its function from fv = b ∧ c to f*v = c
  fv ^ f*v -> DCv

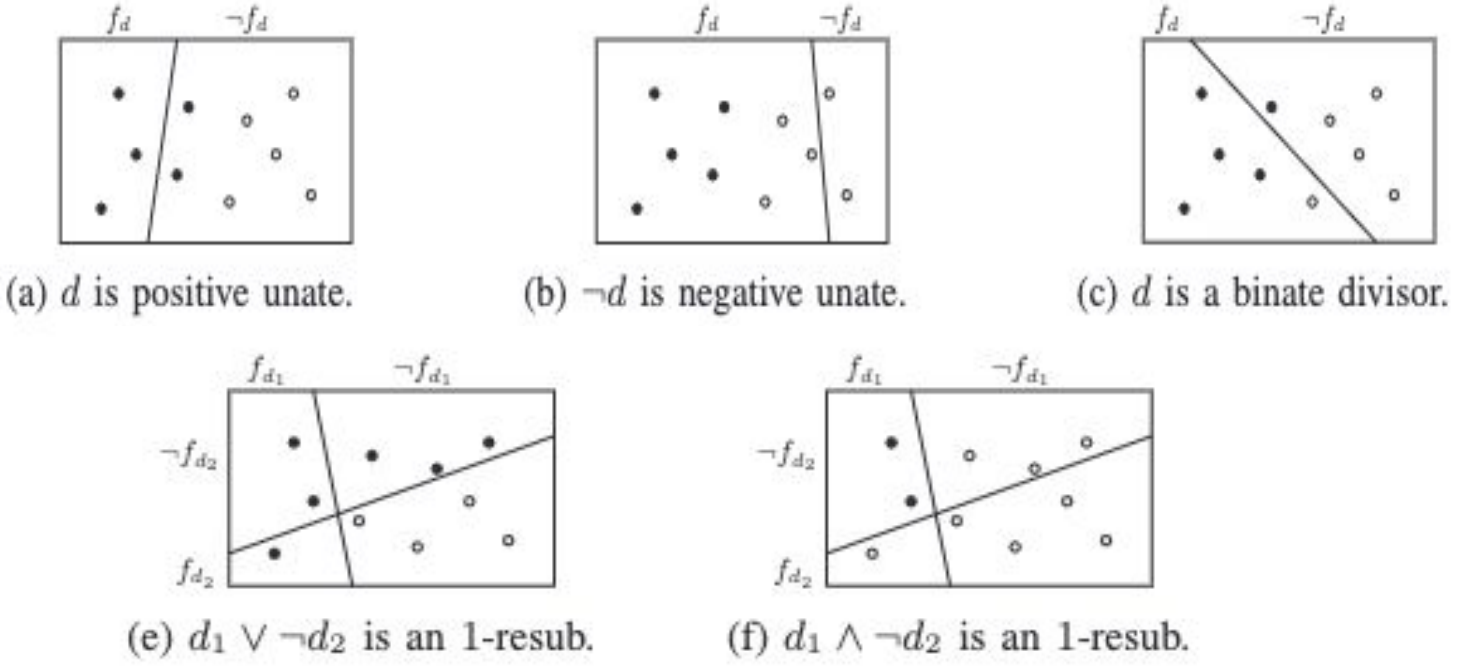


(a) Original sub-graph. (b) Optimized sub-graph.

Fig. 4: Example of ODC-based optimization.

2. Heuristic Boolean resynthesis

Given on-set and off-set functions of Node r and a set D of divisors associated with divisor functions, classify divisors on their intersections with the on-set and the off-set of a target function.



(a) d is positive unate. (b) $\neg d$ is negative unate. (c) d is a binate divisor.

(e) $d_1 \vee \neg d_2$ is an 1-resub. (f) $d_1 \wedge \neg d_2$ is an 1-resub.

Boolean Rewriting Strikes Back: Reconvergence-Driven Windowing Meets Resynthesis

Experimental result for Window Rewriting

case 10-19

10	before : 331 after : 331	15	before : 6194 after : 6189 (-5)
11	before : 3257 after : 3257	16	before : 3029 after : 3029
12	before : 3257 after : 3257	17	before : 33 after : 33
13	before : 142 after : 142	18	before : 19348 after : 19318 (-30)
14	before : 86 after : 86	19	before : 212 after : 205 (-7)

- bigger search path to find window
- resyn with more divisor
- when cost = 0, still update the circuit

case 15

before : 6194
after : 6189

```
--------------------[ DO WINDOW START ]--------------------
AIG[5262] find window
AIG[4278] find window
find 1-resub, negUnate

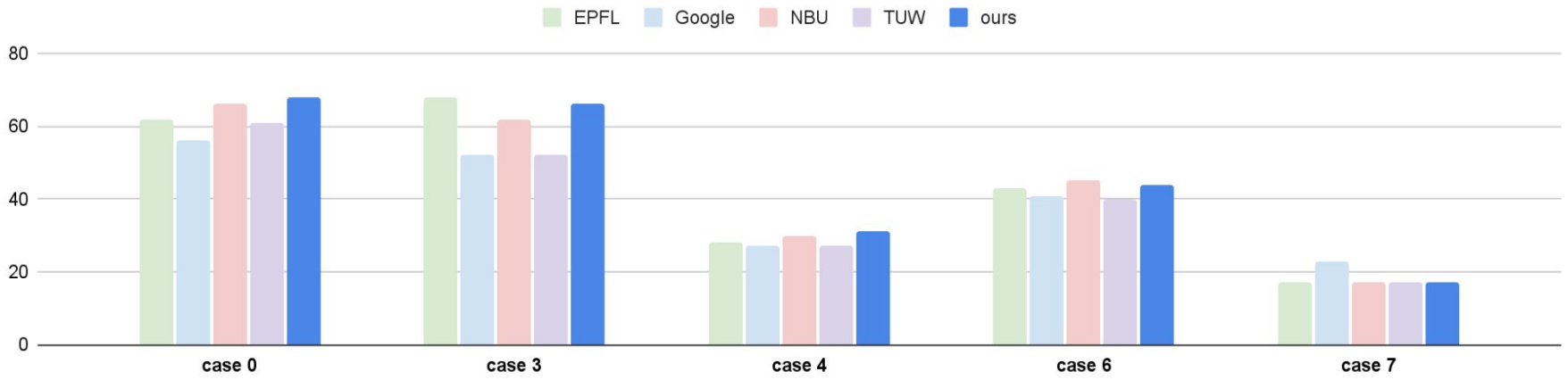
remove 4653
AIG[4278] resyn
AIG[3463] find window
find 2-resub, posUnate
skip: Gs > MFFC
find 1-resub, negUnate

remove 2828
AIG[3464] resyn
AIG[3463] find window
find 2-resub, posUnate
skip: Gs > MFFC
AIG[2928] find window
find 2-resub, negUnate
skip: Gs > MFFC
find 2-resub, posUnate
remove 5083
AIG[2928] resyn
AIG[2906] find window
AIG[2716] find window
find 2-resub, posUnate
skip: Gs > MFFC
find 1-resub, posUnate
remove 5857
AIG[2716] resyn
AIG[1781] find window
AIG[1780] find window
AIG[1779] find window
AIG[ 226] find window
find 1-resub, negUnate

remove 3926
AIG[ 226] resyn
--------------------[ DO WINDOW FINISH ]--------------------
```

Experimental result

each case





sum of nodes



Reference

[1] H. Riener, S. -Y. Lee, A. Mishchenko and G. De Micheli, "Boolean Rewriting Strikes Back: Reconvergence-Driven Windowing Meets Resynthesis," 2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC),

[2] https://people.eecs.berkeley.edu/~alanmi/abc/

Thank You